발췌 : 2026.05.25

개발 실무 입문 교재

학습 교재

Java & Spring Boot 백엔드 입문

Java 객체지향, Spring IoC/DI, Spring Boot, JPA, REST API, 운영 모니터링

Java 개발환경 | 객체지향 | Spring Framework | IoC/DI | Bean 설정 | Spring Boot | JPA | Repository | REST API | Validation | Actuator | DTO/Service

단원 구성

01 Java 개발환경과 객체지향 프로그래밍

02 Spring Framework와 IoC/DI

03 Bean 설정 전략과 어노테이션 기반 DI

04 Spring Boot 시작과 Auto Configuration

05 Profile, Environment, 설정 관리

06 JPA, Hibernate, Spring Data JPA

저작/과정 안내 페이지

본 교안은 교육과정 운영을 위해 기존 차시별 교안을 학습자용 독립 교재 형태로 재구성한 자료다. 실제 수업에서 바로 사용할 수 있도록 목표, 실습, 오류 대응, 보안 유의사항, 확인 문제를 보강했다.

읽기

개념과 용어를 먼저 훑고, 왜 필요한지 수업 맥락을 확인한다.

따라하기

예제 코드를 실행한 뒤 입력값과 조건을 바꾸어 재현한다.

점검하기

오류, 보안 유의사항, 확인 문제로 학습 결과를 검토한다.

대상	개발 입문 및 보안 관점을 함께 익히는 교육생
전제	기본 PC 사용, 명령행 실행, 텍스트 편집, 웹 서비스 이용 경험
산출물	실습 코드, 회고 메모, GitHub 기록

목차

01 Java 개발환경과 객체지향 프로그래밍

02 Spring Framework와 IoC/DI

03 Bean 설정 전략과 어노테이션 기반 DI

04 Spring Boot 시작과 Auto Configuration

05 Profile, Environment, 설정 관리

06 JPA, Hibernate, Spring Data JPA

07 Repository, CRUD 테스트, REST API

08 전역 예외처리, Thymeleaf, Validation

09 Actuator/Admin, DTO/Service, JPA 연관관계

학습 로드맵

1. Java 개발환경과 객체지향 프로그래밍
핵심 활동을 독립 실습 단위로 재구성한다.

2. Spring Framework와 IoC/DI
핵심 활동을 독립 실습 단위로 재구성한다.

3. Bean 설정 전략과 어노테이션 기반 DI
핵심 활동을 독립 실습 단위로 재구성한다.

4. Spring Boot 시작과 Auto Configuration
핵심 활동을 독립 실습 단위로 재구성한다.

5. Profile, Environment, 설정 관리
핵심 활동을 독립 실습 단위로 재구성한다.

6. JPA, Hibernate, Spring Data JPA
핵심 활동을 독립 실습 단위로 재구성한다.

7. Repository, CRUD 테스트, REST API
핵심 활동을 독립 실습 단위로 재구성한다.

8. 전역 예외처리, Thymeleaf, Validation
핵심 활동을 독립 실습 단위로 재구성한다.

9. Actuator/Admin, DTO/Service, JPA 연관관계
핵심 활동을 독립 실습 단위로 재구성한다.

전체 실습 안내

각 장의 코드는 그대로 복사해 실행하는 것보다 입력값, 환경값, 예외 상황을 바꾸어 실험하는 방식으로 학습한다. 모든 실습은 실행 결과와 오류 로그를 함께 기록해야 한다.

1단계	2단계	3단계	4단계
문제 이해	예제 실행	오류 수정	프로젝트 적용

Chapter 01

UNIT Java 개발환경과 객체지향 프로그래밍

학습 주제	자바 언어의 이해 및 기본 문법	수업 흐름	개념 이해· 따라하기· 점검· 적용
산출물	실습 코드, 오류 기록, 확인 문제 답안		

학습 목표

1. Java 객체지향 개념을 백엔드 구조와 연결한다
2. Spring IoC/DI와 Bean 설정 방식을 설명한다
3. Java 개발환경과 객체지향 프로그래밍의 핵심 작업을 실습 절차로 재현한다
4. 개발환경 설정 (JDK8, STS(Spring Tool Suite))
5. 클래스와 생성자

개념	실습	점검	적용
핵심 용어 이해	예제 실행	오류·보안 확인	프로젝트 연결

왜 이 내용을 배우는가

학습 맥락

백엔드 애플리케이션은 객체 모델, 의존성 관리, 데이터 영속성, API 계약, 예외 처리, 운영 모니터링이 함께 맞물려야 안정적으로 동작한다. 이 장에서는 'Java 개발환경과 객체지향 프로그래밍'를 실제 교육 현장에서 바로 따라갈 수 있는 작업 단위로 정리한다.

핵심 개념

- 개발환경 설정 (JDK8, STS(Spring Tool Suite))
- 클래스와 생성자
- 인터페이스와 추상클래스
- 개발환경 설정을 위해 JDK 8 설치 및 환경 구성 절차를 안내하고, STS(Spring Tool Suite) 기반 프로젝트 생성/실행 흐름을 소개
- 객체지향(OOP) 핵심 개념(객체, 클래스, 캡슐화, 책임 분리)을 중심으로 자바 프로그램 구조를 설명

- 클래스와 생성자의 역할(객체 생성, 초기화) 및 생성자 오버로딩 개념을 정리하여 객체 생성 흐름을 이해하도록 지원

주요 용어 정리

IoC

객체 생성과 흐름 제어의 주도권을 프레임워크가 갖는 구조

DI

필요한 의존 객체를 외부에서 주입받아 결합도를 낮추는 방식

Entity

데이터베이스 테이블과 매핑되는 JPA 도메인 객체

Repository

데이터 접근 코드를 추상화해 CRUD와 쿼리 작성을 돕는 계층

따라하기 실습

실습 순서

- 학습 주제 '자바 언어의 이해 및 기본 문법'에 필요한 개발 환경과 예제 폴더를 준비한다.
- 예제 입력과 기대 출력, 실패 조건을 먼저 적어 실습의 완료 기준을 정한다.
- 본문 코드 예제를 실행한 뒤 한 가지 조건을 바꾸어 결과 변화를 확인한다.
- 실습 결과, 오류 로그, 수정 내용을 짧은 회고 메모로 남긴다.

코드 예제

```
public class Member {
    private final String email;
    private String name;

    public Member(String email, String name) {
        this.email = email;
        this.name = name;
    }
}
```

자주 발생하는 오류

오류 예방 체크

- 환경변수, 실행 경로, 포트 번호처럼 로컬 환경에 의존하는 값을 코드에 고정한다.
- 에러 메시지를 읽기 전에 코드를 무작정 수정해 원인을 추적하기 어렵게 만든다.
- 입력 데이터 형식이나 API 응답 구조를 확인하지 않고 후속 처리부터 작성한다.
- Controller에 비즈니스 로직이 몰려 테스트와 재사용이 어려워진다.

보안 관점에서의 유의사항

보안 메모

REST API는 입력 검증, 인증/인가, 예외 응답 일관성, DB 접근 범위를 함께 설계해야 한다. JPA 연관관계는 과도한 노출과 N+1 조회 위험을 점검한다.

실무 적용 예시

개발환경 설정을 위해 JDK 8 설치 및 환경 구성 절차를 안내하고, STS(Spring Tool Suite) 기반 프로젝트 생성/실행 흐름을 소개 / 객체지향(OOP) 핵심 개념(객체, 클래스, 캡슐화, 책임 분리)을 중심으로 자바 프로그램 구조를 설명 / 클래스와 생성자의 역할(객체 생성, 초기화) 및 생성자 오버로딩 개념을 정리하여 객체 생성 흐름을 이해하도록 지원

상황	적용 방법	확인 기준
개별 실습	예제 코드를 실행하고 입력 조건을 변경한다.	결과와 로그를 설명할 수 있다.
팀 프로젝트	역할, 산출물, 리뷰 기준을 정해 저장소에 기록한다.	동료가 같은 절차로 재현할 수 있다.
운영 전 점검	환경값, 오류 처리, 보안 주의사항을 체크리스트로 확인한다.	실패 시 원인과 대응 방법이 문서화되어 있다.

정리하기

이 장의 핵심은 **Java 개발환경과 객체지향 프로그래밍**를 개념 암기에서 끝내지 않고, 입력·처리·검증·공유 가능한 작업 흐름으로 만드는 것이다.

- 개발환경 설정 (JDK8, STS(Spring Tool Suite))
- 클래스와 생성자
- 인터페이스와 추상클래스

확인 문제

1. Java 개발환경과 객체지향 프로그래밍에서 가장 먼저 확인해야 할 입력 또는 요구사항은 무엇인가?
2. 실습 코드가 실패했을 때 어떤 순서로 원인을 좁혀 갈 수 있는가?
3. 이 장의 내용을 팀 프로젝트에 적용한다면 산출물은 무엇이어야 하는가?

다음 학습과의 연결

다음 장에서는 Spring Framework와 IoC/DI로 확장한다.

Chapter 02

UNIT Spring Framework와 IoC/DI

학습 주제	Spring Framework	수업 흐름	개념 이해· 따라하기· 점검· 적용
산출물	실습 코드, 오류 기록, 확인 문제 답안		

학습 목표

1. Java 객체지향 개념을 백엔드 구조와 연결한다
2. Spring IoC/DI와 Bean 설정 방식을 설명한다
3. Spring Framework와 IoC/DI의 핵심 작업을 실습 절차로 재현한다
4. 프레임워크의 개념
5. 라이브러리와 프레임워크의 차이점 이해하기

개념	실습	점검	적용
핵심 용어 이해	예제 실행	오류·보안 확인	프로젝트 연결

왜 이 내용을 배우는가

학습 맥락

백엔드 애플리케이션은 객체 모델, 의존성 관리, 데이터 영속성, API 계약, 예외 처리, 운영 모니터링이 함께 맞물려야 안정적으로 동작한다. 이 장에서는 'Spring Framework와 IoC/DI'를 실제 교육 현장에서 바로 따라갈 수 있는 작업 단위로 정리한다.

핵심 개념

- 프레임워크의 개념
- 라이브러리와 프레임워크의 차이점 이해하기
- IoC(Inversion of Control)
- DI(Dependency Injection) 개념 이해하기
- 프레임워크의 개념을 소개하고, 애플리케이션 개발에서 프레임워크가 제공하는 역할(구조/규칙/흐름 제어)을 중심으로 설명

- 라이브러리와 프레임워크의 차이를 "제어 흐름의 주체" 관점으로 비교하여 이해를 돕고, 활용 사례를 통해 적용 상황을 정리

주요 용어 정리

IoC

객체 생성과 흐름 제어의 주도권을 프레임워크가 갖는 구조

DI

필요한 의존 객체를 외부에서 주입받아 결합도를 낮추는 방식

Entity

데이터베이스 테이블과 매핑되는 JPA 도메인 객체

Repository

데이터 접근 코드를 추상화해 CRUD와 쿼리 작성을 돕는 계층

따라하기 실습

실습 순서

- 학습 주제 'Spring Framework'에 필요한 개발 환경과 예제 폴더를 준비한다.
- 예제 입력과 기대 출력, 실패 조건을 먼저 적어 실습의 완료 기준을 정한다.
- 본문 코드 예제를 실행한 뒤 한 가지 조건을 바꾸어 결과 변화를 확인한다.
- 실습 결과, 오류 로그, 수정 내용을 짧은 회고 메모로 남긴다.

코드 예제

```
@Service
public class OrderService {
    private final PaymentClient paymentClient;

    public OrderService(PaymentClient paymentClient) {
        this.paymentClient = paymentClient;
    }
}
```

자주 발생하는 오류

오류 예방 체크

- 환경변수, 실행 경로, 포트 번호처럼 로컬 환경에 의존하는 값을 코드에 고정한다.

- 에러 메시지를 읽기 전에 코드를 무작정 수정해 원인을 추적하기 어렵게 만든다.
- 입력 데이터 형식이나 API 응답 구조를 확인하지 않고 후속 처리부터 작성한다.
- Controller에 비즈니스 로직이 몰려 테스트와 재사용이 어려워진다.

보안 관점에서의 유의사항

보안 메모

REST API는 입력 검증, 인증/인가, 예외 응답 일관성, DB 접근 범위를 함께 설계해야 한다. JPA 연관관계는 과도한 노출과 N+1 조회 위험을 점검한다.

실무 적용 예시

프레임워크의 개념을 소개하고, 애플리케이션 개발에서 프레임워크가 제공하는 역할(구조/규칙/흐름 제어) 을 중심으로 설명 / 라이브러리와 프레임워크의 차이를 "제어 흐름의 주체" 관점으로 비교하여 이해를 돕고, 활용 사례를 통해 적용 상황을 정리 / IoC(Inversion of Control) 개념을 설명하고, 객체 생성/관리의 책임이 개발자 코드에서 컨테이너로 이동하는 의미를 안내

상황	적용 방법	확인 기준
개별 실습	예제 코드를 실행하고 입력 조건을 변경한다.	결과와 로그를 설명할 수 있다.
팀 프로젝트	역할, 산출물, 리뷰 기준을 정해 저장소에 기록한다.	동료가 같은 절차로 재현할 수 있다.
운영 전 점검	환경값, 오류 처리, 보안 주의사항을 체크리스트로 확인한다.	실패 시 원인과 대응 방법이 문서화되어 있다.

정리하기

이 장의 핵심은 **Spring Framework와 IoC/DI**를 개념 암기에서 끝내지 않고, 입력·처리·검증·공유 가능한작업 흐름으로 만드는 것이다.

- 프레임워크의 개념
- 라이브러리와 프레임워크의 차이점 이해하기
- IoC(Inversion of Control)

확인 문제

1. Spring Framework와 IoC/DI에서 가장 먼저 확인해야 할 입력 또는 요구사항은 무엇인가?
2. 실습 코드가 실패했을 때 어떤 순서로 원인을 좁혀 갈 수 있는가?
3. 이 장의 내용을 팀 프로젝트에 적용한다면 산출물은 무엇이어야 하는가?

다음 학습과의 연결

다음 장에서는 Bean 설정 전략과 어노테이션 기반 DI로 확장한다.

학습주제: Spring Framework

Chapter 03

UNIT Bean 설정 전략과 어노테이션 기반 DI

학습 주제	Spring Framework	수업 흐름	개념 이해· 따라하기· 점검· 적용
산출물	실습 코드, 오류 기록, 확인 문제 답안		

학습 목표

1. Java 객체지향 개념을 백엔드 구조와 연결한다
2. Spring IoC/DI와 Bean 설정 방식을 설명한다
3. Bean 설정 전략과 어노테이션 기반 DI의 핵심 작업을 실습 절차로 재현한다
4. DI 전략1 과 전략2
5. Spring Bean을 모두 XML로 설정하기

개념	실습	점검	적용
핵심 용어 이해	예제 실행	오류·보안 확인	프로젝트 연결

왜 이 내용을 배우는가

학습 맥락

백엔드 애플리케이션은 객체 모델, 의존성 관리, 데이터 영속성, API 계약, 예외 처리, 운영 모니터링이 함께 맞물려야 안정적으로 동작한다. 이 장에서는 'Bean 설정 전략과 어노테이션 기반 DI'를 실제 교육 현장에서 바로 따라갈 수 있는 작업 단위로 정리한다.

핵심 개념

- DI 전략1 과 전략2
- Spring Bean을 모두 XML로 설정하기
- Spring Bean을 XML로 설정하기와 어노테이션을 선언 및 사용하기
- @Component @Autowired @Resource 이해하기
- DI 전략 1과 전략 2의 개념을 정리하고, 의존성 주입 방식에 따른 구성 차이와 적용 흐름을 비교 설명

- Spring Bean을 XML로 설정하는 방식(Bean 등록, id/name, class 지정, 의존 관계 연결)의 구조를 안내하고 설정 파일 구성 원리를 설명

주요 용어 정리

IoC

객체 생성과 흐름 제어의 주도권을 프레임워크가 갖는 구조

DI

필요한 의존 객체를 외부에서 주입받아 결합도를 낮추는 방식

Entity

데이터베이스 테이블과 매핑되는 JPA 도메인 객체

Repository

데이터 접근 코드를 추상화해 CRUD와 쿼리 작성을 돕는 계층

따라하기 실습

실습 순서

- 학습 주제 'Spring Framework'에 필요한 개발 환경과 예제 폴더를 준비한다.
- 예제 입력과 기대 출력, 실패 조건을 먼저 적어 실습의 완료 기준을 정한다.
- 본문 코드 예제를 실행한 뒤 한 가지 조건을 바꾸어 결과 변화를 확인한다.
- 실습 결과, 오류 로그, 수정 내용을 짧은 회고 메모로 남긴다.

코드 예제

```
@Configuration
public class AppConfig {
    @Bean
    public Clock clock() {
        return Clock.systemDefaultZone();
    }
}
```

자주 발생하는 오류

오류 예방 체크

- 환경변수, 실행 경로, 포트 번호처럼 로컬 환경에 의존하는 값을 코드에 고정한다.

- 에러 메시지를 읽기 전에 코드를 무작정 수정해 원인을 추적하기 어렵게 만든다.
- 입력 데이터 형식이나 API 응답 구조를 확인하지 않고 후속 처리부터 작성한다.
- Controller에 비즈니스 로직이 몰려 테스트와 재사용이 어려워진다.

보안 관점에서의 유의사항

보안 메모

REST API는 입력 검증, 인증/인가, 예외 응답 일관성, DB 접근 범위를 함께 설계해야 한다. JPA 연관관계는 과도한 노출과 N+1 조회 위험을 점검한다.

실무 적용 예시

DI 전략 1과 전략 2의 개념을 정리하고, 의존성 주입 방식에 따른 구성 차이와 적용 흐름을 비교 설명 / Spring Bean을 XML로 설정하는 방식(Bean 등록, id/name, class 지정, 의존 관계 연결)의 구조를 안내하고 설정 파일 구성 원리를 설명 / XML 기반 설정과 어노테이션 기반 설정을 함께 소개하여, 각각의 장단점 (명시성/유연성 vs 편의성/자동화) 및 혼합 구성 시의 흐름을 정리

상황	적용 방법	확인 기준
개별 실습	예제 코드를 실행하고 입력 조건을 변경한다.	결과와 로그를 설명할 수 있다.
팀 프로젝트	역할, 산출물, 리뷰 기준을 정해 저장소에 기록한다.	동료가 같은 절차로 재현할 수 있다.
운영 전 점검	환경값, 오류 처리, 보안 주의사항을 체크리스트로 확인한다.	실패 시 원인과 대응 방법이 문서화되어 있다.

정리하기

이 장의 핵심은 **Bean 설정 전략과 어노테이션 기반 DI**를 개념 암기에서 끝내지 않고, 입력·처리·검증·공유 가능한 작업 흐름으로 만드는 것이다.

- DI 전략1 과 전략2
- Spring Bean을 모두 XML로 설정하기
- Spring Bean을 XML로 설정하기와 어노테이션을 선언 및 사용하기

확인 문제

1. Bean 설정 전략과 어노테이션 기반 DI에서 가장 먼저 확인해야 할 입력 또는 요구사항은 무엇인가?
2. 실습 코드가 실패했을 때 어떤 순서로 원인을 좁혀 갈 수 있는가?
3. 이 장의 내용을 팀 프로젝트에 적용한다면 산출물은 무엇이어야 하는가?

다음 학습과의 연결

다음 장에서는 Spring Boot 시작과 Auto Configuration로 확장한다.

학습주제: Spring Framework

Chapter 04

UNIT Spring Boot 시작과 Auto Configuration

학습 주제	Spring Framework Spring Boot	수업 흐름	개념 이해· 따라하기· 점검· 적용
산출물	실습 코드, 오류 기록, 확인 문제 답안		

학습 목표

1. Java 객체지향 개념을 백엔드 구조와 연결한다
2. Spring IoC/DI와 Bean 설정 방식을 설명한다
3. Spring Boot 시작과 Auto Configuration의 핵심 작업을 실습 절차로 재현한다
4. Spring Bean을 어노테이션을 선언과 Config 클래스 작성
5. @Configuration 과 @Bean 이해하기

개념	실습	점검	적용
핵심 용어 이해	예제 실행	오류·보안 확인	프로젝트 연결

왜 이 내용을 배우는가

학습 맥락

백엔드 애플리케이션은 객체 모델, 의존성 관리, 데이터 영속성, API 계약, 예외 처리, 운영 모니터링이 함께 맞물려야 안정적으로 동작한다. 이 장에서는 'Spring Boot 시작과 Auto Configuration'를 실제 교육 현장에서 바로 따라갈 수 있는 작업 단위로 정리한다.

핵심 개념

- Spring Bean을 어노테이션을 선언과 Config 클래스 작성
- @Configuration 과 @Bean 이해하기
- 개발환경 설정 (JDK17, IntelliJ IDEA)
- Spring Boot 의존성관리와 Auto Configuration
- Spring Boot Application 의 역할 이해하기
- DI 전략 3의 개념을 정리하고, 기존 DI 방식들과의 차이 및 적용 흐름을 비교하여 설명

주요 용어 정리

IoC

객체 생성과 흐름 제어의 주도권을 프레임워크가 갖는 구조

DI

필요한 의존 객체를 외부에서 주입받아 결합도를 낮추는 방식

Entity

데이터베이스 테이블과 매핑되는 JPA 도메인 객체

Repository

데이터 접근 코드를 추상화해 CRUD와 쿼리 작성을 돕는 계층

따라하기 실습

실습 순서

- 학습 주제 'Spring Framework Spring Boot'에 필요한 개발 환경과 예제 폴더를 준비한다.
- 예제 입력과 기대 출력, 실패 조건을 먼저 적어 실습의 완료 기준을 정한다.
- 본문 코드 예제를 실행한 뒤 한 가지 조건을 바꾸어 결과 변화를 확인한다.
- 실습 결과, 오류 로그, 수정 내용을 짧은 회고 메모로 남긴다.

코드 예제

```
@SpringBootApplication
public class RookiesApplication {
    public static void main(String[] args) {
        SpringApplication.run(RookiesApplication.class, args);
    }
}
```

자주 발생하는 오류

오류 예방 체크

- 환경변수, 실행 경로, 포트 번호처럼 로컬 환경에 의존하는 값을 코드에 고정한다.
- 에러 메시지를 읽기 전에 코드를 무작정 수정해 원인을 추적하기 어렵게 만든다.
- 입력 데이터 형식이나 API 응답 구조를 확인하지 않고 후속 처리부터 작성한다.
- Controller에 비즈니스 로직이 몰려 테스트와 재사용이 어려워진다.

보안 관점에서의 유의사항

> **보안 메모**
>
> REST API는 입력 검증, 인증/인가, 예외 응답 일관성, DB 접근 범위를 함께 설계해야 한다. JPA 연관관계는 과도한 노출과 N+1 조회 위험을 점검한다.

실무 적용 예시

DI 전략 3의 개념을 정리하고, 기존 DI 방식들과의 차이 및 적용 흐름을 비교하여 설명 / 어노테이션 기반 구성과 함께 Config 클래스(Java Config) 작성 방식을 소개하고, 설정을 코드로 관리하는 목적과 장점을 안내 / @Configuration의 역할(설정 클래스 선언, 컨테이너 구성 정보 제공)을 설명하고, 설정 클래스가 Bean을 관리하는 구조를 정리

상황	적용 방법	확인 기준
개별 실습	예제 코드를 실행하고 입력 조건을 변경한다.	결과와 로그를 설명할 수 있다.
팀 프로젝트	역할, 산출물, 리뷰 기준을 정해 저장소에 기록한다.	동료가 같은 절차로 재현할 수 있다.
운영 전 점검	환경값, 오류 처리, 보안 주의사항을 체크리스트로 확인한다.	실패 시 원인과 대응 방법이 문서화되어 있다.

정리하기

이 장의 핵심은 **Spring Boot 시작과 Auto Configuration**를 개념 암기에서 끝내지 않고, 입력·처리·검증·공유 가능한 작업 흐름으로 만드는 것이다.

- Spring Bean을 어노테이션을 선언과 Config 클래스 작성
- @Configuration 과 @Bean 이해하기
- 개발환경 설정 (JDK17, IntelliJ IDEA)

확인 문제

1. Spring Boot 시작과 Auto Configuration에서 가장 먼저 확인해야 할 입력 또는 요구사항은 무엇인가?
2. 실습 코드가 실패했을 때 어떤 순서로 원인을 좁혀 갈 수 있는가?
3. 이 장의 내용을 팀 프로젝트에 적용한다면 산출물은 무엇이어야 하는가?

다음 학습과의 연결

다음 장에서는 Profile, Environment, 설정 관리로 확장한다.

학습주제: Spring Framework Spring Boot

Chapter 05

UNIT Profile, Environment, 설정 관리

학습 주제	Spring Boot	수업 흐름	개념 이해· 따라하기· 점검· 적용
산출물	실습 코드, 오류 기록, 확인 문제 답안		

학습 목표

1. Java 객체지향 개념을 백엔드 구조와 연결한다
2. Spring IoC/DI와 Bean 설정 방식을 설명한다
3. Profile, Environment, 설정 관리의 핵심 작업을 실습 절차로 재현한다
4. ApplicationRunner 구현하기
5. Profile 과 Environment 이해하기

개념	실습	점검	적용
핵심 용어 이해	예제 실행	오류·보안 확인	프로젝트 연결

왜 이 내용을 배우는가

학습 맥락

백엔드 애플리케이션은 객체 모델, 의존성 관리, 데이터 영속성, API 계약, 예외 처리, 운영 모니터링이 함께 맞물려야 안정적으로 동작한다. 이 장에서는 'Profile, Environment, 설정 관리'를 실제 교육 현장에서 바로 따라갈 수 있는 작업 단위로 정리한다.

핵심 개념

- ApplicationRunner 구현하기
- Profile 과 Environment 이해하기
- Environment 환경별 설정파일 작성하기
- 사용자정의 Properties 클래스 작성하기
- `@Profile 과 spring.profile.active 속성 이해하기

- ApplicationRunner의 역할(애플리케이션 구동 시점 실행 로직)과 구현 방식(인터페이스 구현, 실행 흐름) 을 소개

주요 용어 정리

IoC

객체 생성과 흐름 제어의 주도권을 프레임워크가 갖는 구조

DI

필요한 의존 객체를 외부에서 주입받아 결합도를 낮추는 방식

Entity

데이터베이스 테이블과 매핑되는 JPA 도메인 객체

Repository

데이터 접근 코드를 추상화해 CRUD와 쿼리 작성을 돕는 계층

따라하기 실습

실습 순서

- 학습 주제 'Spring Boot'에 필요한 개발 환경과 예제 폴더를 준비한다.
- 예제 입력과 기대 출력, 실패 조건을 먼저 적어 실습의 완료 기준을 정한다.
- 본문 코드 예제를 실행한 뒤 한 가지 조건을 바꾸어 결과 변화를 확인한다.
- 실습 결과, 오류 로그, 수정 내용을 짧은 회고 메모로 남긴다.

코드 예제

```
@ConfigurationProperties(prefix = "app")
public record AppProperties(String uploadDir, int pageSize) {}

@Profile("local")
@Bean
CommandLineRunner seedData(MemberRepository repository) {
    return args -> repository.save(new Member("test@example.com"));
}
```

자주 발생하는 오류

오류 예방 체크

- 환경변수, 실행 경로, 포트 번호처럼 로컬 환경에 의존하는 값을 코드에 고정한다.

- 에러 메시지를 읽기 전에 코드를 무작정 수정해 원인을 추적하기 어렵게 만든다.
- 입력 데이터 형식이나 API 응답 구조를 확인하지 않고 후속 처리부터 작성한다.
- Controller에 비즈니스 로직이 몰려 테스트와 재사용이 어려워진다.

보안 관점에서의 유의사항

보안 메모

REST API는 입력 검증, 인증/인가, 예외 응답 일관성, DB 접근 범위를 함께 설계해야 한다. JPA 연관관계는 과도한 노출과 N+1 조회 위험을 점검한다.

실무 적용 예시

ApplicationRunner의 역할(애플리케이션 구동 시점 실행 로직)과 구현 방식(인터페이스 구현, 실행 흐름)을 소개 / 실행 시점에 수행할 초기화 작업(데이터 세팅, 점검 로직 등)에서 ApplicationRunner가 활용되는 이유를 설명 / Profile 개념을 통해 실행 환경(로컬/개발/운영 등)을 분리하는 목적과 필요성을 안내

상황	적용 방법	확인 기준
개별 실습	예제 코드를 실행하고 입력 조건을 변경한다.	결과와 로그를 설명할 수 있다.
팀 프로젝트	역할, 산출물, 리뷰 기준을 정해 저장소에 기록한다.	동료가 같은 절차로 재현할 수 있다.
운영 전 점검	환경값, 오류 처리, 보안 주의사항을 체크리스트로 확인한다.	실패 시 원인과 대응 방법이 문서화되어 있다.

정리하기

이 장의 핵심은 **Profile, Environment, 설정 관리**를 개념 암기에서 끝내지 않고, 입력·처리·검증·공유가능한 작업 흐름으로 만드는 것이다.

- ApplicationRunner 구현하기
- Profile 과 Environment 이해하기
- Environment 환경별 설정파일 작성하기

확인 문제

1. Profile, Environment, 설정 관리에서 가장 먼저 확인해야 할 입력 또는 요구사항은 무엇인가?

2. 실습 코드가 실패했을 때 어떤 순서로 원인을 좁혀 갈 수 있는가?
3. 이 장의 내용을 팀 프로젝트에 적용한다면 산출물은 무엇이어야 하는가?

다음 학습과의 연결

다음 장에서는 JPA, Hibernate, Spring Data JPA로 확장한다.

학습주제: Spring Boot

Chapter 06

UNIT JPA, Hibernate, Spring Data JPA

학습 주제	Spring Boot	수업 흐름	개념 이해· 따라하기· 점검· 적용
산출물	실습 코드, 오류 기록, 확인 문제 답안		

학습 목표

1. Java 객체지향 개념을 백엔드 구조와 연결한다
2. Spring IoC/DI와 Bean 설정 방식을 설명한다
3. JPA, Hibernate, Spring Data JPA의 핵심 작업을 실습 절차로 재현한다
4. "- ORM 을 위한 JPA 와 Hibernate 와 Spring Data JPA 역할 이해하기
5. Entity 작성하기

개념	실습	점검	적용
핵심 용어 이해	예제 실행	오류·보안 확인	프로젝트 연결

왜 이 내용을 배우는가

학습 맥락

백엔드 애플리케이션은 객체 모델, 의존성 관리, 데이터 영속성, API 계약, 예외 처리, 운영 모니터링이 함께 맞물려야 안정적으로 동작한다. 이 장에서는 'JPA, Hibernate, Spring Data JPA'를 실제 교육 현장에서 바로 따라갈 수 있는 작업 단위로 정리한다.

핵심 개념

- "- ORM 을 위한 JPA 와 Hibernate 와 Spring Data JPA 역할 이해하기
- Entity 작성하기
- GenerationType의 4가지 속성의 차이점 이해하기
- ddl-auto 의 4가지 속성(create, create-drop, update, validate) 이해하기"
- ORM 관점에서 JPA, Hibernate, Spring Data JPA의 역할과 관계를 정리하여 설명
- JPA: ORM 표준(명세)

주요 용어 정리

IoC

객체 생성과 흐름 제어의 주도권을 프레임워크가 갖는 구조

DI

필요한 의존 객체를 외부에서 주입받아 결합도를 낮추는 방식

Entity

데이터베이스 테이블과 매핑되는 JPA 도메인 객체

Repository

데이터 접근 코드를 추상화해 CRUD와 쿼리 작성을 돕는 계층

FetchType

연관 엔티티를 즉시 또는 지연 로딩할지 결정하는 설정

따라하기 실습

실습 순서

- 학습 주제 'Spring Boot'에 필요한 개발 환경과 예제 폴더를 준비한다.
- 예제 입력과 기대 출력, 실패 조건을 먼저 적어 실습의 완료 기준을 정한다.
- 본문 코드 예제를 실행한 뒤 한 가지 조건을 바꾸어 결과 변화를 확인한다.
- 실습 결과, 오류 로그, 수정 내용을 짧은 회고 메모로 남긴다.

코드 예제

```
@Entity
public class Post {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;
}
```

자주 발생하는 오류

오류 예방 체크

- 환경변수, 실행 경로, 포트 번호처럼 로컬 환경에 의존하는 값을 코드에 고정한다.

- 에러 메시지를 읽기 전에 코드를 무작정 수정해 원인을 추적하기 어렵게 만든다.
- 입력 데이터 형식이나 API 응답 구조를 확인하지 않고 후속 처리부터 작성한다.
- Controller에 비즈니스 로직이 몰려 테스트와 재사용이 어려워진다.

보안 관점에서의 유의사항

보안 메모

REST API는 입력 검증, 인증/인가, 예외 응답 일관성, DB 접근 범위를 함께 설계해야 한다. JPA 연관관계는 과도한 노출과 N+1 조회 위험을 점검한다.

실무 적용 예시

ORM 관점에서 JPA, Hibernate, Spring Data JPA의 역할과 관계를 정리하여 설명 / JPA: ORM 표준(명세) / Hibernate: JPA 구현체(실제 동작 엔진)

상황	적용 방법	확인 기준
개별 실습	예제 코드를 실행하고 입력 조건을 변경한다.	결과와 로그를 설명할 수 있다.
팀 프로젝트	역할, 산출물, 리뷰 기준을 정해 저장소에 기록한다.	동료가 같은 절차로 재현할 수 있다.
운영 전 점검	환경값, 오류 처리, 보안 주의사항을 체크리스트로 확인한다.	실패 시 원인과 대응 방법이 문서화되어 있다.

정리하기

이 장의 핵심은 **JPA, Hibernate, Spring Data JPA**를 개념 암기에서 끝내지 않고, 입력·처리·검증·공유가능한 작업 흐름으로 만드는 것이다.

- "- ORM 을 위한 JPA 와 Hibernate 와 Spring Data JPA 역할 이해하기
- Entity 작성하기
- GenerationType의 4가지 속성의 차이점 이해하기

확인 문제

1. JPA, Hibernate, Spring Data JPA에서 가장 먼저 확인해야 할 입력 또는 요구사항은 무엇인가?

2. 실습 코드가 실패했을 때 어떤 순서로 원인을 좁혀 갈 수 있는가?

3. 이 장의 내용을 팀 프로젝트에 적용한다면 산출물은 무엇이어야 하는가?

다음 학습과의 연결

다음 장에서는 Repository, CRUD 테스트, REST API로 확장한다.

Chapter 07

UNIT Repository, CRUD 테스트, REST API

학습 주제	Spring Boot	수업 흐름	개념 이해· 따라하기· 점검· 적용
산출물	실습 코드, 오류 기록, 확인 문제 답안		

학습 목표

1. Java 객체지향 개념을 백엔드 구조와 연결한다
2. Spring IoC/DI와 Bean 설정 방식을 설명한다
3. Repository, CRUD 테스트, REST API의 핵심 작업을 실습 절차로 재현한다
4. JPARepository의 메서드 사용법 이해하기
5. Query Method의 규칙 이해하기

개념	실습	점검	적용
핵심 용어 이해	예제 실행	오류·보안 확인	프로젝트 연결

왜 이 내용을 배우는가

학습 맥락

백엔드 애플리케이션은 객체 모델, 의존성 관리, 데이터 영속성, API 계약, 예외 처리, 운영 모니터링이 함께 맞물려야 안정적으로 동작한다. 이 장에서는 'Repository, CRUD 테스트, REST API'를 실제 교육 현장에서 바로 따라갈 수 있는 작업 단위로 정리한다.

핵심 개념

- JPARepository의 메서드 사용법 이해하기
- Query Method의 규칙 이해하기
- SpringBootTest 클래스 작성하여 CRUD 테스트하기
- Spring MVC 이해하기
- REST API 작성하기

- JpaRepository의 주요 메서드 사용법(CRUD 관련 기본 제공 메서드, 페이징/정렬 지원)을 개념 중심으로 정리하고 활용 흐름을 설명

주요 용어 정리

IoC

객체 생성과 흐름 제어의 주도권을 프레임워크가 갖는 구조

DI

필요한 의존 객체를 외부에서 주입받아 결합도를 낮추는 방식

Entity

데이터베이스 테이블과 매핑되는 JPA 도메인 객체

Repository

데이터 접근 코드를 추상화해 CRUD와 쿼리 작성을 돕는 계층

따라하기 실습

실습 순서

- 학습 주제 'Spring Boot'에 필요한 개발 환경과 예제 폴더를 준비한다.
- 예제 입력과 기대 출력, 실패 조건을 먼저 적어 실습의 완료 기준을 정한다.
- 본문 코드 예제를 실행한 뒤 한 가지 조건을 바꾸어 결과 변화를 확인한다.
- 실습 결과, 오류 로그, 수정 내용을 짧은 회고 메모로 남긴다.

코드 예제

```
@SpringBootTest
class MemberRepositoryTest {
    @Autowired MemberRepository repository;

    @Test
    void saveAndFind() {
        Member saved = repository.save(new Member("a@b.com"));
        assertThat(repository.findById(saved.getId())).isPresent();
    }
}
```

자주 발생하는 오류

오류 예방 체크

- 환경변수, 실행 경로, 포트 번호처럼 로컬 환경에 의존하는 값을 코드에 고정한다.
- 에러 메시지를 읽기 전에 코드를 무작정 수정해 원인을 추적하기 어렵게 만든다.
- 입력 데이터 형식이나 API 응답 구조를 확인하지 않고 후속 처리부터 작성한다.
- Controller에 비즈니스 로직이 몰려 테스트와 재사용이 어려워진다.

보안 관점에서의 유의사항

보안 메모

REST API는 입력 검증, 인증/인가, 예외 응답 일관성, DB 접근 범위를 함께 설계해야 한다. JPA 연관관계는 과도한 노출과 N+1 조회 위험을 점검한다.

실무 적용 예시

JpaRepository의 주요 메서드 사용법(CRUD 관련 기본 제공 메서드, 페이징/정렬 지원)을 개념 중심으로 정리하고 활용 흐름을 설명 / 메서드 이름으로 쿼리를 생성하는 Query Method 규칙을 안내하고, 조건/정렬/ 조합 방식의 네이밍 패턴을 이해하도록 지원 / @SpringBootTest 기반 테스트 클래스 작성 목적과 구조를 소개하고, CRUD 테스트 시 테스트 흐름(준비 → 실행 → 검증)을 설명

상황	적용 방법	확인 기준
개별 실습	예제 코드를 실행하고 입력 조건을 변경한다.	결과와 로그를 설명할 수 있다.
팀 프로젝트	역할, 산출물, 리뷰 기준을 정해 저장소에 기록한다.	동료가 같은 절차로 재현할 수 있다.
운영 전 점검	환경값, 오류 처리, 보안 주의사항을 체크리스트로 확인한다.	실패 시 원인과 대응 방법이 문서화되어 있다.

정리하기

이 장의 핵심은 **Repository, CRUD 테스트, REST API**를 개념 암기에서 끝내지 않고, 입력·처리·검증·공유 가능한 작업 흐름으로 만드는 것이다.

- JPARepository의 메서드 사용법 이해하기
- Query Method의 규칙 이해하기

- SpringBootTest 클래스 작성하여 CRUD 테스트하기

확인 문제

1. Repository, CRUD 테스트, REST API에서 가장 먼저 확인해야 할 입력 또는 요구사항은 무엇인가?
2. 실습 코드가 실패했을 때 어떤 순서로 원인을 좁혀 갈 수 있는가?
3. 이 장의 내용을 팀 프로젝트에 적용한다면 산출물은 무엇이어야 하는가?

다음 학습과의 연결

다음 장에서는 전역 예외처리, Thymeleaf, Validation로 확장한다.

Chapter 08

UNIT 전역 예외처리, Thymeleaf, Validation

학습 주제	Spring Boot	수업 흐름	개념 이해· 따라하기· 점검· 적용
산출물	실습 코드, 오류 기록, 확인 문제 답안		

학습 목표

1. Java 객체지향 개념을 백엔드 구조와 연결한다
2. Spring IoC/DI와 Bean 설정 방식을 설명한다
3. 전역 예외처리, Thymeleaf, Validation의 핵심 작업을 실습 절차로 재현한다
4. REST API 작성하기, 전역 예외처리 Advice 작성하기
5. Thymeleaf 를 사용하여 서버사이드렌더링 페이지 작성하기

개념	실습	점검	적용
핵심 용어 이해	예제 실행	오류·보안 확인	프로젝트 연결

왜 이 내용을 배우는가

학습 맥락

백엔드 애플리케이션은 객체 모델, 의존성 관리, 데이터 영속성, API 계약, 예외 처리, 운영 모니터링이 함께 맞물려야 안정적으로 동작한다. 이 장에서는 '전역 예외처리, Thymeleaf, Validation'를 실제 교육 현장에서 바로 따라갈 수 있는 작업 단위로 정리한다.

핵심 개념

- REST API 작성하기, 전역 예외처리 Advice 작성하기
- Thymeleaf 를 사용하여 서버사이드렌더링 페이지 작성하기
- 입력항목 검증을 위한 Bean Validation 이해하기
- Spring 기반으로 REST API 작성 흐름(Controller 설계, 요청/응답 구조, HTTP 메서드 매핑)을 정리하고 구현 관점을 안내

- 예외 상황을 일관된 형식으로 처리하기 위한 전역 예외처리(Advice) 개념을 소개하고, 예외를 중앙에서 핸들링하는 목적(일관된 응답, 중복 제거, 유지보수성)을 설명
- Thymeleaf를 활용한 서버사이드 렌더링(SSR) 방식의 페이지 작성 구조를 안내하고, 템플릿 엔진의 동작 흐름(컨트롤러 모델 전달 → 템플릿 렌더링)을 설명

주요 용어 정리

IoC

객체 생성과 흐름 제어의 주도권을 프레임워크가 갖는 구조

DI

필요한 의존 객체를 외부에서 주입받아 결합도를 낮추는 방식

Entity

데이터베이스 테이블과 매핑되는 JPA 도메인 객체

Repository

데이터 접근 코드를 추상화해 CRUD와 쿼리 작성을 돕는 계층

따라하기 실습

실습 순서

- 학습 주제 'Spring Boot'에 필요한 개발 환경과 예제 폴더를 준비한다.
- 예제 입력과 기대 출력, 실패 조건을 먼저 적어 실습의 완료 기준을 정한다.
- 본문 코드 예제를 실행한 뒤 한 가지 조건을 바꾸어 결과 변화를 확인한다.
- 실습 결과, 오류 로그, 수정 내용을 짧은 회고 메모로 남긴다.

코드 예제

```
@RestControllerAdvice
public class ApiExceptionHandler {
    @ExceptionHandler(MethodArgumentNotValidException.class)
    ResponseEntity<Map<String, String>> invalid(MethodArgumentNotValidException ex) {
        return ResponseEntity.badRequest().body(Map.of("message", "입력값을 확인하세요"));
    }
}
```

자주 발생하는 오류

오류 예방 체크

- 환경변수, 실행 경로, 포트 번호처럼 로컬 환경에 의존하는 값을 코드에 고정한다.
- 에러 메시지를 읽기 전에 코드를 무작정 수정해 원인을 추적하기 어렵게 만든다.
- 입력 데이터 형식이나 API 응답 구조를 확인하지 않고 후속 처리부터 작성한다.
- Controller에 비즈니스 로직이 몰려 테스트와 재사용이 어려워진다.

보안 관점에서의 유의사항

보안 메모

REST API는 입력 검증, 인증/인가, 예외 응답 일관성, DB 접근 범위를 함께 설계해야 한다. JPA 연관관계는 과도한 노출과 N+1 조회 위험을 점검한다.

실무 적용 예시

Spring 기반으로 REST API 작성 흐름(Controller 설계, 요청/응답 구조, HTTP 메서드 매핑)을 정리하고 구현 관점을 안내 / 예외 상황을 일관된 형식으로 처리하기 위한 전역 예외처리(Advice) 개념을 소개하고, 예외를 중앙에서 핸들링하는 목적(일관된 응답, 중복 제거, 유지보수성)을 설명 / Thymeleaf를 활용한 서버사이드 렌더링(SSR) 방식의 페이지 작성 구조를 안내하고, 템플릿 엔진의 동작 흐름(컨트롤러 모델 전달 → 템플릿 렌더링)을 설명

상황	적용 방법	확인 기준
개별 실습	예제 코드를 실행하고 입력 조건을 변경한다.	결과와 로그를 설명할 수 있다.
팀 프로젝트	역할, 산출물, 리뷰 기준을 정해 저장소에 기록한다.	동료가 같은 절차로 재현할 수 있다.
운영 전 점검	환경값, 오류 처리, 보안 주의사항을 체크리스트로 확인한다.	실패 시 원인과 대응 방법이 문서화되어 있다.

정리하기

이 장의 핵심은 **전역 예외처리, Thymeleaf, Validation**를 개념 암기에서 끝내지 않고, 입력·처리·검증·공유 가능한 작업 흐름으로 만드는 것이다.

- REST API 작성하기, 전역 예외처리 Advice 작성하기
- Thymeleaf 를 사용하여 서버사이드렌더링 페이지 작성하기

- 입력항목 검증을 위한 Bean Validation 이해하기

확인 문제

1. 전역 예외처리, Thymeleaf, Validation에서 가장 먼저 확인해야 할 입력 또는 요구사항은 무엇인가?
2. 실습 코드가 실패했을 때 어떤 순서로 원인을 좁혀 갈 수 있는가?
3. 이 장의 내용을 팀 프로젝트에 적용한다면 산출물은 무엇이어야 하는가?

다음 학습과의 연결

다음 장에서는 Actuator/Admin, DTO/Service, JPA 연관관계로 확장한다.

학습주제: Spring Boot

Chapter 09

UNIT Actuator/Admin, DTO/Service, JPA 연관관계

학습 주제	Spring Boot	수업 흐름	개념 이해· 따라하기· 점검· 적용
산출물	실습 코드, 오류 기록, 확인 문제 답안		

학습 목표

1. Java 객체지향 개념을 백엔드 구조와 연결한다
2. Spring IoC/DI와 Bean 설정 방식을 설명한다
3. Actuator/Admin, DTO/Service, JPA 연관관계의 핵심 작업을 실습 절차로 재현한다
4. Spring Boot Actuator 및 Spring Boot Admin으로 모니터링 페이지 작성하기
5. Service 와 DTO(Data Transfer Object) 를 추가하여 REST API 구조 변경하기

개념	실습	점검	적용
핵심 용어 이해	예제 실행	오류·보안 확인	프로젝트 연결

왜 이 내용을 배우는가

학습 맥락

백엔드 애플리케이션은 객체 모델, 의존성 관리, 데이터 영속성, API 계약, 예외 처리, 운영 모니터링이 함께 맞물려야 안정적으로 동작한다. 이 장에서는 'Actuator/Admin, DTO/Service, JPA 연관관계'를 실제 교육 현장에서 바로 따라갈 수 있는 작업 단위로 정리한다.

핵심 개념

- Spring Boot Actuator 및 Spring Boot Admin으로 모니터링 페이지 작성하기
- Service 와 DTO(Data Transfer Object) 를 추가하여 REST API 구조 변경하기
- JPA 연관관계 일대일관계 @JoinColumn 사용하기
- FetchType.LAZY, FetchType.EAGER 개념 및 장단점 비교하기

- Spring Boot Actuator의 역할을 설명하고, 애플리케이션의 상태 정보, 헬스 체크, 메트릭, 환경 정보 등을 모니터링할 수 있는 기본 구조를 안내하였습니다.
- Spring Boot Admin을 활용하여 여러 애플리케이션의 Actuator 정보를 한 곳에서 확인할 수 있는 모니터링 페이지 구성 방식을 설명하였습니다.

주요 용어 정리

IoC

객체 생성과 흐름 제어의 주도권을 프레임워크가 갖는 구조

DI

필요한 의존 객체를 외부에서 주입받아 결합도를 낮추는 방식

Entity

데이터베이스 테이블과 매핑되는 JPA 도메인 객체

Repository

데이터 접근 코드를 추상화해 CRUD와 쿼리 작성을 돕는 계층

FetchType

연관 엔티티를 즉시 또는 지연 로딩할지 결정하는 설정

따라하기 실습

실습 순서

- 학습 주제 'Spring Boot'에 필요한 개발 환경과 예제 폴더를 준비한다.
- 예제 입력과 기대 출력, 실패 조건을 먼저 적어 실습의 완료 기준을 정한다.
- 본문 코드 예제를 실행한 뒤 한 가지 조건을 바꾸어 결과 변화를 확인한다.
- 실습 결과, 오류 로그, 수정 내용을 짧은 회고 메모로 남긴다.

코드 예제

```
@Service
@Transactional
public class PostService {
    public PostResponse create(PostCreateRequest request) {
        Post post = postRepository.save(request.toEntity());
        return PostResponse.from(post);
```

```
    }
}
```

자주 발생하는 오류

오류 예방 체크

- 환경변수, 실행 경로, 포트 번호처럼 로컬 환경에 의존하는 값을 코드에 고정한다.
- 에러 메시지를 읽기 전에 코드를 무작정 수정해 원인을 추적하기 어렵게 만든다.
- 입력 데이터 형식이나 API 응답 구조를 확인하지 않고 후속 처리부터 작성한다.
- Controller에 비즈니스 로직이 몰려 테스트와 재사용이 어려워진다.

보안 관점에서의 유의사항

보안 메모

REST API는 입력 검증, 인증/인가, 예외 응답 일관성, DB 접근 범위를 함께 설계해야 한다. JPA 연관관계는 과도한 노출과 N+1 조회 위험을 점검한다.

실무 적용 예시

Spring Boot Actuator의 역할을 설명하고, 애플리케이션의 상태 정보, 헬스 체크, 메트릭, 환경 정보 등을 모니터링할 수 있는 기본 구조를 안내하였습니다. / Spring Boot Admin을 활용하여 여러 애플리케이션의 Actuator 정보를 한 곳에서 확인할 수 있는 모니터링 페이지 구성 방식을 설명하였습니다. / 운영 관점에서 모니터링의 필요성을 강조하고, 장애 징후 확인, 상태 점검, 서비스 가시성 확보 측면에서 Actuator와 Admin의 활용 목적을 정리하였습니다.

상황	적용 방법	확인 기준
개별 실습	예제 코드를 실행하고 입력 조건을 변경한다.	결과와 로그를 설명할 수 있다.
팀 프로젝트	역할, 산출물, 리뷰 기준을 정해 저장소에 기록한다.	동료가 같은 절차로 재현할 수 있다.
운영 전 점검	환경값, 오류 처리, 보안 주의사항을 체크리스트로 확인한다.	실패 시 원인과 대응 방법이 문서화되어 있다.

정리하기

이 장의 핵심은 **Actuator/Admin, DTO/Service, JPA 연관관계**를 개념 암기에서 끝내지 않고, 입력·처리·검증·공유 가능한 작업 흐름으로 만드는 것이다.

- Spring Boot Actuator 및 Spring Boot Admin으로 모니터링 페이지 작성하기
- Service 와 DTO(Data Transfer Object) 를 추가하여 REST API 구조 변경하기
- JPA 연관관계 일대일관계 @JoinColumn 사용하기

확인 문제

1. Actuator/Admin, DTO/Service, JPA 연관관계에서 가장 먼저 확인해야 할 입력 또는 요구사항은
2. 무엇인가?
3. 실습 코드가 실패했을 때 어떤 순서로 원인을 좁혀 갈 수 있는가?
4. 이 장의 내용을 팀 프로젝트에 적용한다면 산출물은 무엇이어야 하는가?

다음 학습과의 연결

이 장의 내용을 부록 체크리스트로 종합한다.

학습주제: Spring Boot

정리 페이지

Java & Spring Boot 백엔드 입문의 목적은 도구 사용법을 외우는 것이 아니라, 문제를 정의하고 재현 가능한 산출물로 만드는 습관을 갖추는 것이다.

- 학습 전: 목표와 완료 기준을 적는다.
- 학습 중: 실행 결과와 오류 로그를 기록한다.
- 학습 후: 동료가 재현할 수 있는 문서와 저장소 상태로 정리한다.

부록 또는 참고 자료

- Java/Spring 개발환경 체크리스트
- DI 방식 비교표
- JPA 어노테이션 참조
- REST API 응답 규칙